

Trouve ton artisan
Plateforme de mise en relation avec les
artisans
de la région Auvergne-Rhône-Alpes

Lien GitHub :

<https://github.com/adripcr0301-sketch/trouve-ton-artisan>

Site en ligne :

<https://animated-taiyaki-e08cf4.netlify.app>

Maquettes Figma :

<https://www.figma.com/design/9XxcE4RcEiPT5Nr6tNJzdU/Trouve-ton-artisan>

1. Contexte du projet	3
2. Maquettes Figma	5
3. Base de données	8
4. Sécurité	11
5. Veille sécurité	14

1. CONTEXTE DU PROJET

Présentation de l'entreprise

La Région Auvergne-Rhône-Alpes est une collectivité territoriale française regroupant 12 départements et près de 8 millions d'habitants. Elle dispose d'une direction numérique chargée de développer des services en ligne à destination des citoyens et des acteurs économiques locaux.

Besoins identifiés

La Région souhaite développer une plateforme web permettant aux particuliers de trouver facilement un artisan de confiance sur son territoire. Les artisans référencés exercent dans quatre secteurs : Bâtiment, Alimentation, Fabrication et Services.

Les besoins fonctionnels sont les suivants :

Consulter les artisans par catégorie de métier

Rechercher un artisan par son nom

Consulter la fiche détaillée d'un artisan (note, spécialité, ville, présentation)

Contacteur un artisan directement via un formulaire en ligne

Contraintes techniques

Frontend : React.js + Bootstrap + Sass

Backend : Node.js + Express + Sequelize

Base de données : MySQL

Design : charte graphique de la Région Auvergne-Rhône-Alpes

Responsive : mobile-first, compatible mobile / tablette / desktop

Accessibilité : conformité WCAG 2.1

Hébergement en ligne obligatoire

Livrables

Livrable Détail

Maquettes Figma 5 pages × 3 formats + prototypage

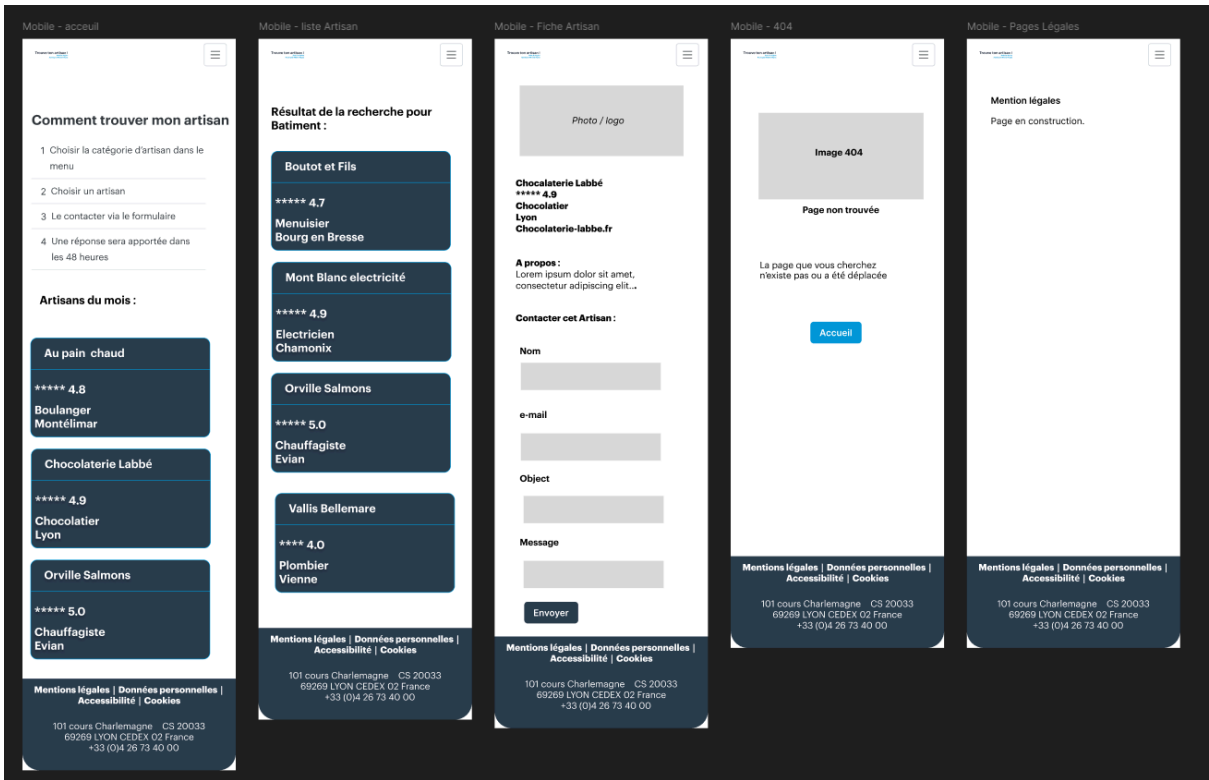
Dépôt GitHub Code source versionné avec branches et PR

Site en ligne Frontend Netlify + API Render + BDD Clever Cloud

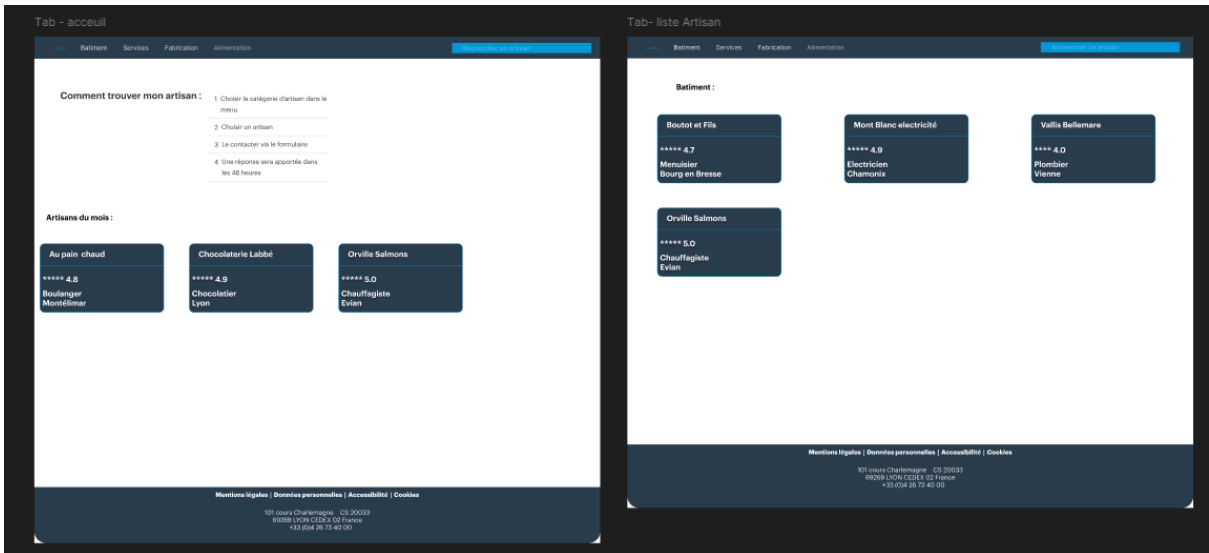
Dossier PDF Documentation complète du projet

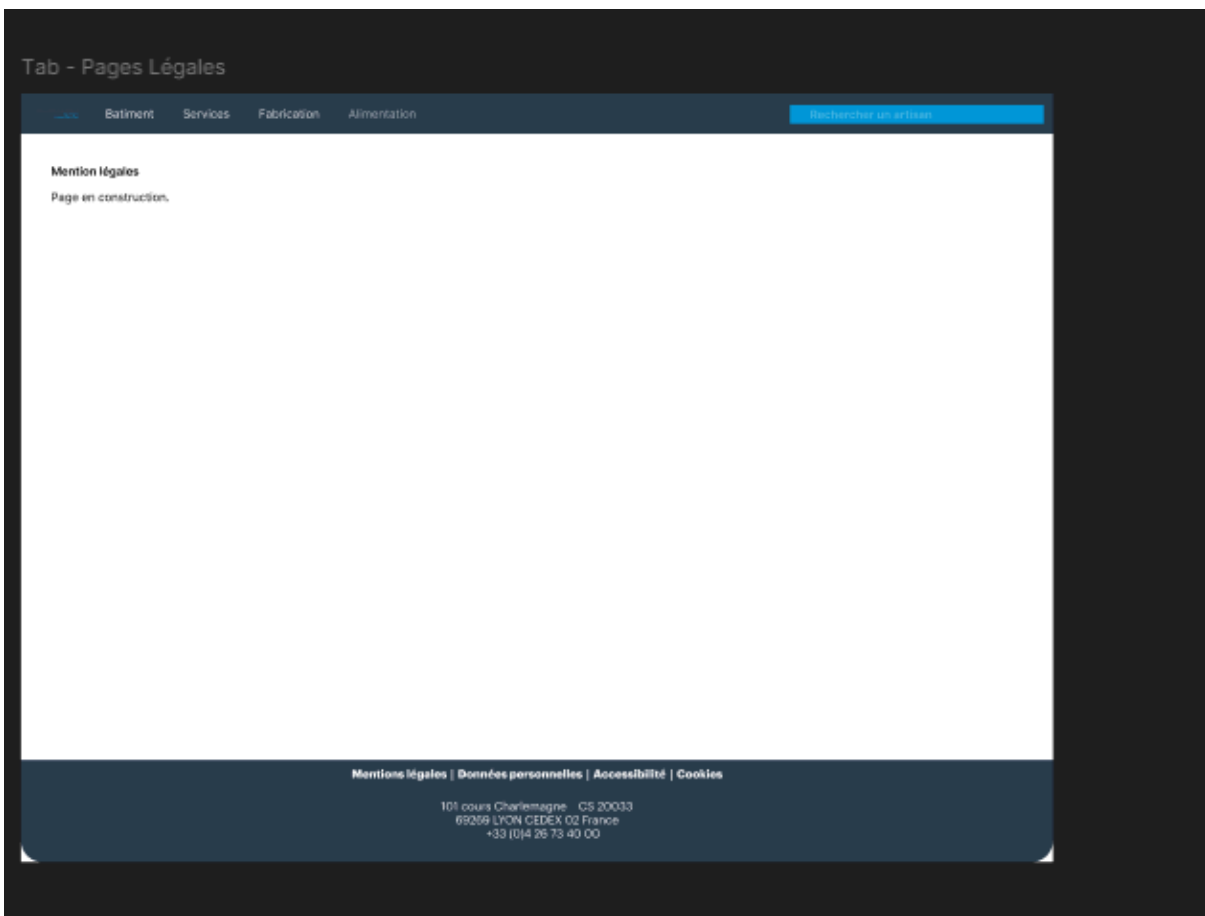
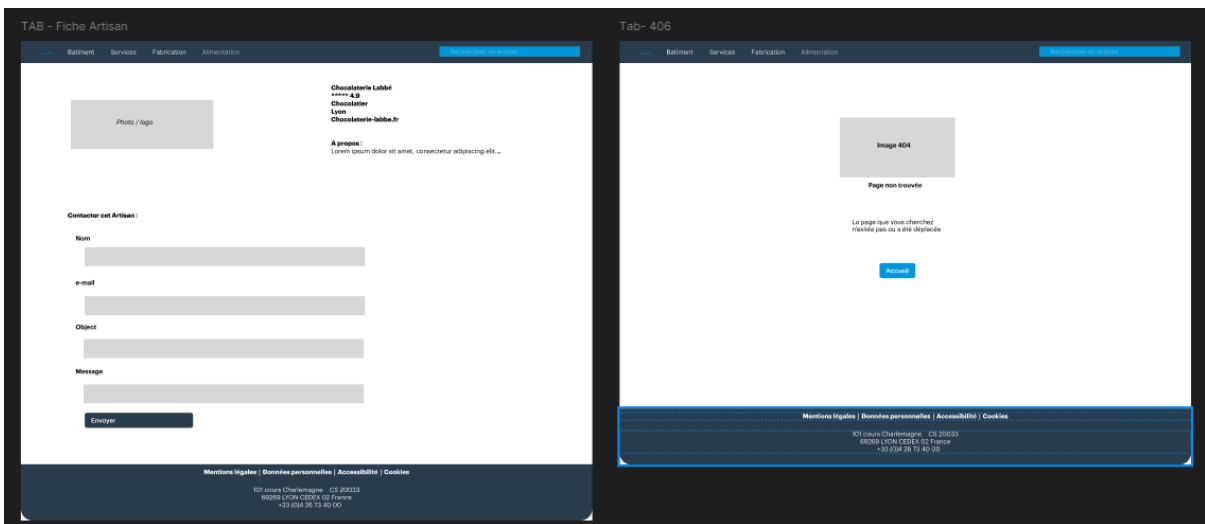
2. MAQUETTES FIGMA

Maquette mobile:

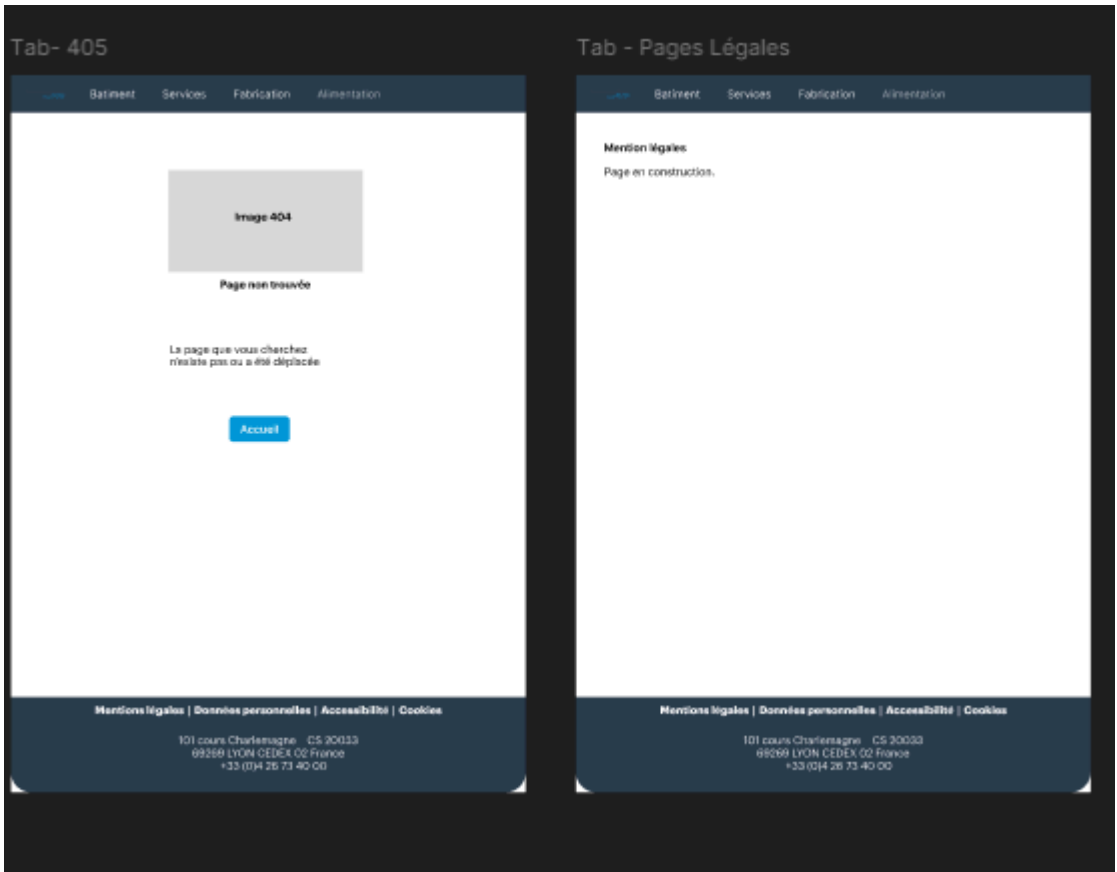
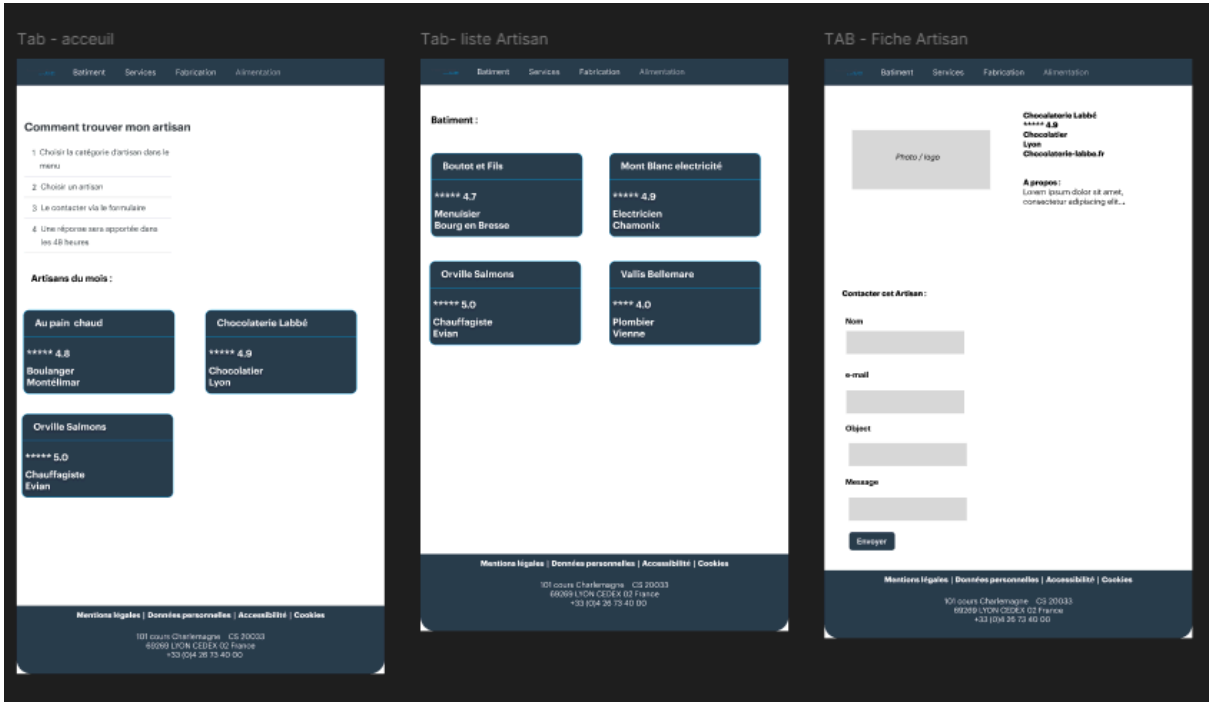


Maquette Desktop :





Maquette Tablette :



Structure à respecter :

Page d'accueil : mobile / tablette / desktop

Liste artisans : mobile / tablette / desktop

Fiche artisan : mobile / tablette / desktop

Page 404 : mobile

Page légale : mobile

Lien vers les maquettes complètes :

<https://www.figma.com/design/9XxcE4RcEiPT5Nr6tNJzdU/Trouve-ton-artisan>

3. BASE DE DONNÉES

Modèle Conceptuel de Données (MCD)

CATEGORIE ----< SPECIALITE ----< ARTISAN

Une catégorie contient plusieurs spécialités. Une spécialité regroupe plusieurs artisans.

Modèle Logique de Données (MLD):

categorie (id, nom)

specialite (id, nom, #id_categorie)

artisan (id, nom, note, ville, a_propos, email, site_web, image, top, #id_specialite)

Description des tables

Table categorie

Les 4 catégories : Alimentation, Bâtiment, Fabrication, Services.

Champ	type	description
id	INT UNSIGNED	Identifiant unique (clé primaire)
nom	VARCHAR(100)	Nom de la catégorie (unique)

Table specialite

Champ	Type	Description
id	INT UNSIGNED	Identifiant unique (clé primaire)
nom	VARCHAR(100)	Nom de la spécialité
id_categorie	INT UNSIGNED	Clé étrangère vers categorie

Table artisan

Champ	Type	Description
id	INT UNSIGNED	Identifiant unique (clé primaire)
nom	VARCHAR(150)	Nom de l'artisan ou de l'entreprise
note	DECIMAL(2,1)	Note de 0 à 5
ville	VARCHAR(100)	Ville d'exercice
a_propos	TEXT	Présentation de l'artisan
email	VARCHAR(150)	Adresse email de contact
site_web	VARCHAR(255)	Site web (optionnel)
image	VARCHAR(255)	Photo (optionnelle)
top	TINYINT(1)	Artisan mis en avant (0 ou 1)
id_specialite	INT UNSIGNED	Clé étrangère vers specialite

La base contient 17 artisans répartis dans 15 spécialités et 4 catégories.

4. SÉCURITÉ

Mesures de sécurité mises en place

1. Authentification par clé API (X-API-Key)

Toutes les routes de l'API nécessitent un header X-API-Key contenant une clé secrète. Toute requête sans clé valide reçoit une réponse HTTP 401.

Intérêt : empêche tout accès direct à l'API depuis un navigateur ou un script non autorisé. Seule l'application frontend connaît la clé.

2. CORS (Cross-Origin Resource Sharing)

Le serveur n'accepte les requêtes que depuis l'URL du frontend (Netlify). Toute autre origine est bloquée.

Intérêt : protège l'API contre les requêtes provenant de sites tiers malveillants.

3. Helmet

Helmet est un middleware Express qui configure automatiquement plusieurs en-têtes HTTP de sécurité : Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, etc.

Intérêt : réduit la surface d'attaque en empêchant le clickjacking, l'injection de contenu et d'autres attaques courantes côté navigateur.

4. Limitation de débit (Rate Limiting)

Le serveur limite chaque adresse IP à 100 requêtes par fenêtre de 15 minutes. Au-delà, une réponse HTTP 429 est renvoyée.

Intérêt : protège l'API contre les attaques par déni de service (DoS) et le scraping abusif.

5. Validation des données (express-validator)

Chaque donnée reçue via le formulaire de contact (nom, email, objet, message, artisan_id) est validée et nettoyée avant traitement.

Intérêt : prévient les injections (SQL, XSS) en refusant les données mal formées et en échappant les caractères dangereux.

6. Variables d'environnement (.env)

Aucune information sensible (mots de passe, clés, identifiants de BDD) n'est présente dans le code source. Toutes ces valeurs sont stockées dans un fichier .env exclu du dépôt Git.

Intérêt : évite l'exposition des secrets en cas de publication accidentelle du code.

5. VEILLE SÉCURITÉ

Méthodologie

La veille sécurité a été menée en référence au Top 10 OWASP (Open Web Application Security Project), qui recense les dix vulnérabilités web les plus critiques.

Vulnérabilités étudiées

Injection SQL (A03:2021)

Une injection SQL consiste à insérer du code SQL malveillant dans une entrée utilisateur pour manipuler la base de données.

Recherche effectuée : test avec des valeurs comme ' OR '1'='1 dans les paramètres de l'API.

Résultat : l'utilisation de Sequelize avec des requêtes paramétrées protège naturellement contre ce type d'attaque. Les valeurs ne sont jamais concaténées directement dans les requêtes SQL.

Correction : aucune nécessaire, Sequelize gère la protection par défaut.

Cross-Site Scripting – XSS (A03:2021)

Une attaque XSS consiste à injecter du code JavaScript malveillant dans une page web, qui sera exécuté par le navigateur d'un autre utilisateur.

Recherche effectuée : envoi de <script>alert('xss')</script> via le formulaire de contact.

Résultat : express-validator échappe les caractères HTML dans les données reçues. React échappe également le contenu affiché par défaut.

Correction : validation et nettoyage des entrées avec express-validator, utilisation de React (qui échappe le contenu par défaut).

Exposition de données sensibles (A02:2021)

Les données sensibles (mots de passe BDD, clé API, identifiants SMTP) pourraient être exposées si elles étaient présentes dans le code source versionné.

Recherche effectuée : vérification de l'historique Git et du contenu des fichiers commités.

Résultat : aucun secret trouvé dans le dépôt. Le fichier .env est correctement ignoré via .gitignore.

Correction : toutes les valeurs sensibles sont dans .env, un fichier .env.example documente les variables sans révéler les valeurs.

Défaillance du contrôle d'accès (A01:2021)

Sans protection, n'importe qui pourrait interroger l'API et récupérer les données des artisans.

Recherche effectuée : appel direct aux routes API sans header X-API-Key.

Résultat : le middleware de sécurité renvoie une erreur 401 pour toute requête sans clé valide.

Correction : middleware apiKeyAuth appliqué sur toutes les routes /api/*.